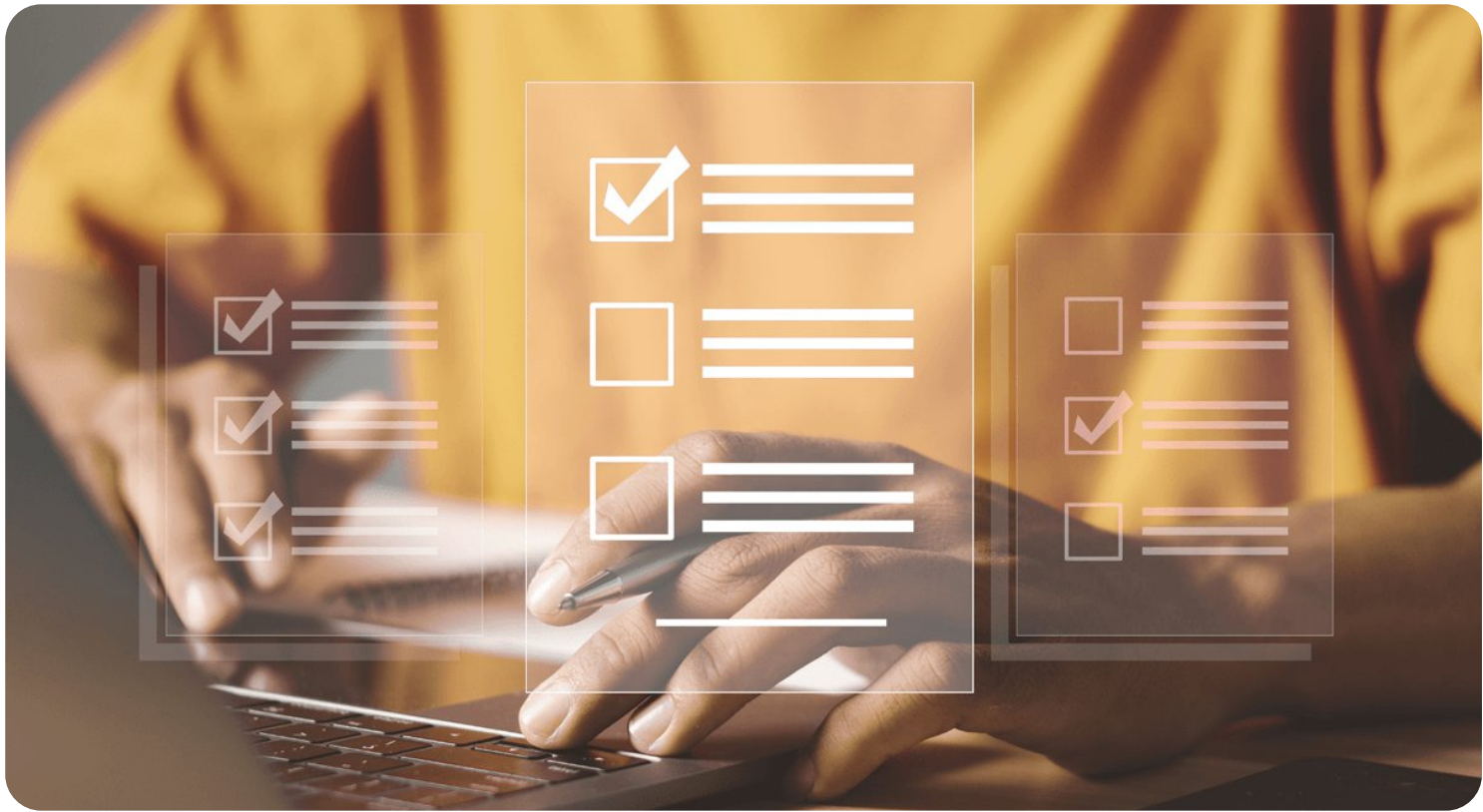




Semana 7

Desarrollo Cloud Native (CDY2204)

Informe escrito

Formato de respuesta

Nombre estudiante:	Natalia Alvarado Egor Llancaichun
Asignatura: Desarrollo Cloud Native	Carrera: Analista Programador
Profesor: Alonso Castillo	Fecha: 06-07-2026

ÍNDICE

1. INTRODUCCIÓN	3
2. OBJETIVOS DE LA IMPLEMENTACIÓN	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. ARQUITECTURA DE LA SOLUCIÓN	4
4. TECNOLOGÍAS UTILIZADAS	5
5. IMPLEMENTACIÓN DE RABBITMQ	6
5.1. Levantamiento del servicio mediante Docker	6
5.2. Configuración y despliegue	7
5.3. Integración con Spring Boot	7
6. PRUEBAS FUNCIONALES EN POSTMAN	8
6.1. Envío de resúmenes	9
6.2. Consulta del estado de la cola	9
7. VALIDACIÓN TÉCNICA EN RABBITMQ	11
8. CONCLUSIONES	11

1. INTRODUCCIÓN

Durante la Semana 7 se integró RabbitMQ al proyecto **LearningPlatformCloud**, continuando el desarrollo realizado durante las semanas anteriores. El objetivo principal fue incorporar un mecanismo de mensajería asíncrona que permitiera desacoplar el envío de los resúmenes de inscripción del procesamiento posterior de la información.

La solución implementada permite que un resumen asociado a una inscripción sea enviado desde la aplicación Spring Boot hacia un exchange de RabbitMQ. El mensaje es posteriormente direccionado mediante una routing key hacia una cola específica, donde permanece disponible hasta que sea procesado por el componente consumidor.

Este enfoque permite separar la recepción de una solicitud del procesamiento posterior de sus datos. De esta forma, la aplicación puede responder al cliente sin depender de que todas las operaciones se ejecuten de manera inmediata dentro de la misma solicitud HTTP.

La implementación se integró con la infraestructura previamente desarrollada, manteniendo la comunicación mediante API Gateway, el despliegue de la aplicación en una instancia EC2, el uso de contenedores Docker y la conexión con Oracle Cloud.

2. OBJETIVOS DE LA IMPLEMENTACIÓN

2.1. Objetivo general

Integrar un servicio de mensajería asíncrona basado en RabbitMQ al proyecto LearningPlatformCloud, permitiendo enviar los resúmenes de inscripción hacia una cola para su posterior procesamiento y persistencia.

2.2. Objetivos específicos

- Configurar RabbitMQ como servicio de mensajería.
- Ejecutar RabbitMQ mediante un contenedor Docker.
- Crear un exchange para recibir los mensajes generados por la aplicación.
- Crear una cola durable para almacenar temporalmente los mensajes.
- Configurar una routing key para direccionar los mensajes.
- Integrar Spring Boot con RabbitMQ.
- Implementar el envío de resúmenes de inscripción hacia la cola.
- Implementar el procesamiento posterior de los mensajes.
- Mantener la integración con Oracle Cloud.
- Desplegar la solución actualizada en una instancia EC2.
- Validar la comunicación entre API Gateway, la aplicación y RabbitMQ.
- Documentar el flujo completo mediante evidencias técnicas.

3. ARQUITECTURA DE LA SOLUCIÓN

La arquitectura implementada mantiene los componentes desarrollados durante las semanas anteriores e incorpora RabbitMQ como intermediario de mensajería.

El flujo general de la solución es el siguiente:

Cliente / Postman → API Gateway → Aplicación Spring Boot en EC2 → Exchange RabbitMQ → Routing Key → Queue RabbitMQ → Consumidor → Oracle Cloud

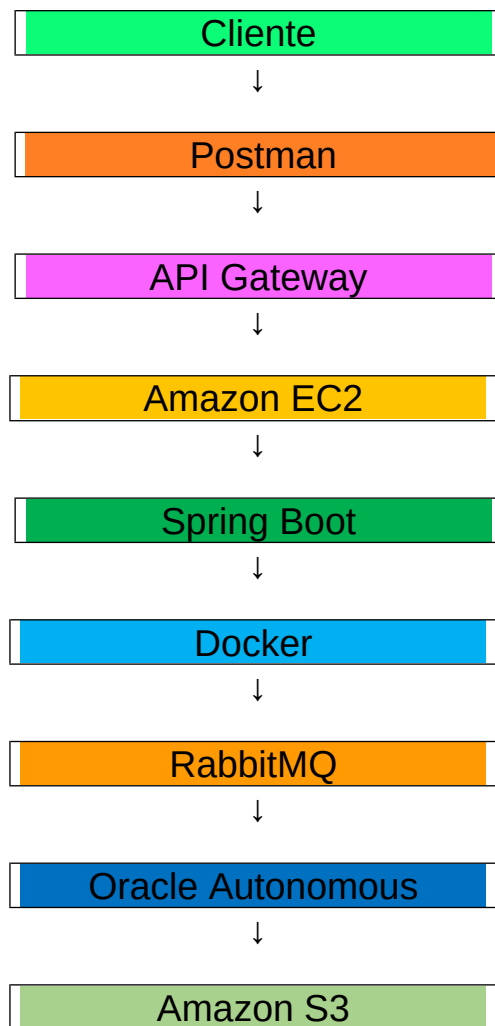
Cuando el cliente solicita el envío de un resumen, la petición ingresa mediante API Gateway y es procesada por la aplicación desplegada en EC2. La aplicación genera el mensaje correspondiente y lo publica en el exchange configurado.

RabbitMQ evalúa la routing key utilizada y dirige el mensaje hacia la cola asociada mediante el binding existente. El mensaje permanece en la cola hasta que el componente consumidor lo procesa.

Los componentes principales de la arquitectura son:

- **Postman:** utilizado para ejecutar y validar las solicitudes HTTP.
- **Amazon API Gateway:** punto de entrada público hacia la API.
- **Amazon EC2:** infraestructura donde se ejecutan los servicios.
- **Spring Boot:** aplicación responsable de la lógica del sistema.
- **Docker:** plataforma utilizada para ejecutar los servicios en contenedores.
- **RabbitMQ:** broker encargado de gestionar la mensajería asíncrona.
- **Oracle Cloud:** base de datos utilizada para la persistencia de la información.
- **Amazon S3:** almacenamiento utilizado por funcionalidades desarrolladas previamente.

Arquitectura General



4. Tecnologías utilizadas

Tecnología	Función
Java 17	Lenguaje principal del backend
Spring Boot	Desarrollo de la API REST
Spring AMQP	Integración entre Spring Boot y RabbitMQ
RabbitMQ	Broker de mensajería asíncrona
Docker	Contenerización de los servicios
Docker Compose	Orquestación de los contenedores
Amazon EC2	Ejecución de la aplicación en la nube
Amazon API Gateway	Exposición de endpoints
Oracle Cloud	Persistencia de información
Amazon S3	Almacenamiento de archivos
Postman	Pruebas funcionales de la API
Git y GitHub	Control de versiones

5. IMPLEMENTACIÓN DE RABBITMQ

5.1. Levantamiento del servicio mediante Docker

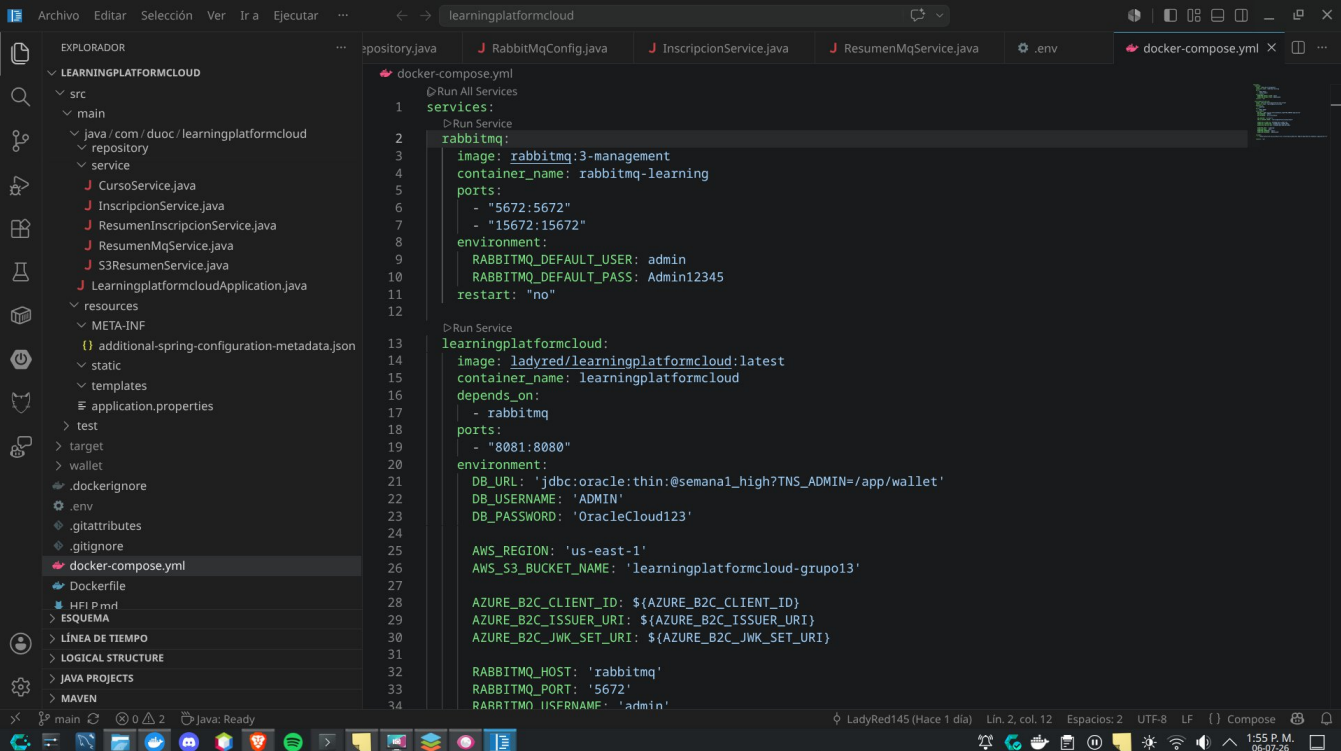
RabbitMQ fue ejecutado como un servicio contenerizado, permitiendo mantener su configuración aislada y reproducible.

El contenedor incorpora la interfaz de administración de RabbitMQ, utilizada para verificar visualmente la creación de los recursos y el flujo de mensajes.

Los principales puertos utilizados son:

- **5672:** comunicación AMQP entre la aplicación y RabbitMQ.
- **15672:** interfaz web de administración de RabbitMQ.

Para la administración remota se utilizó un túnel SSH hacia la instancia EC2, evitando depender del acceso directo permanente al panel administrativo.

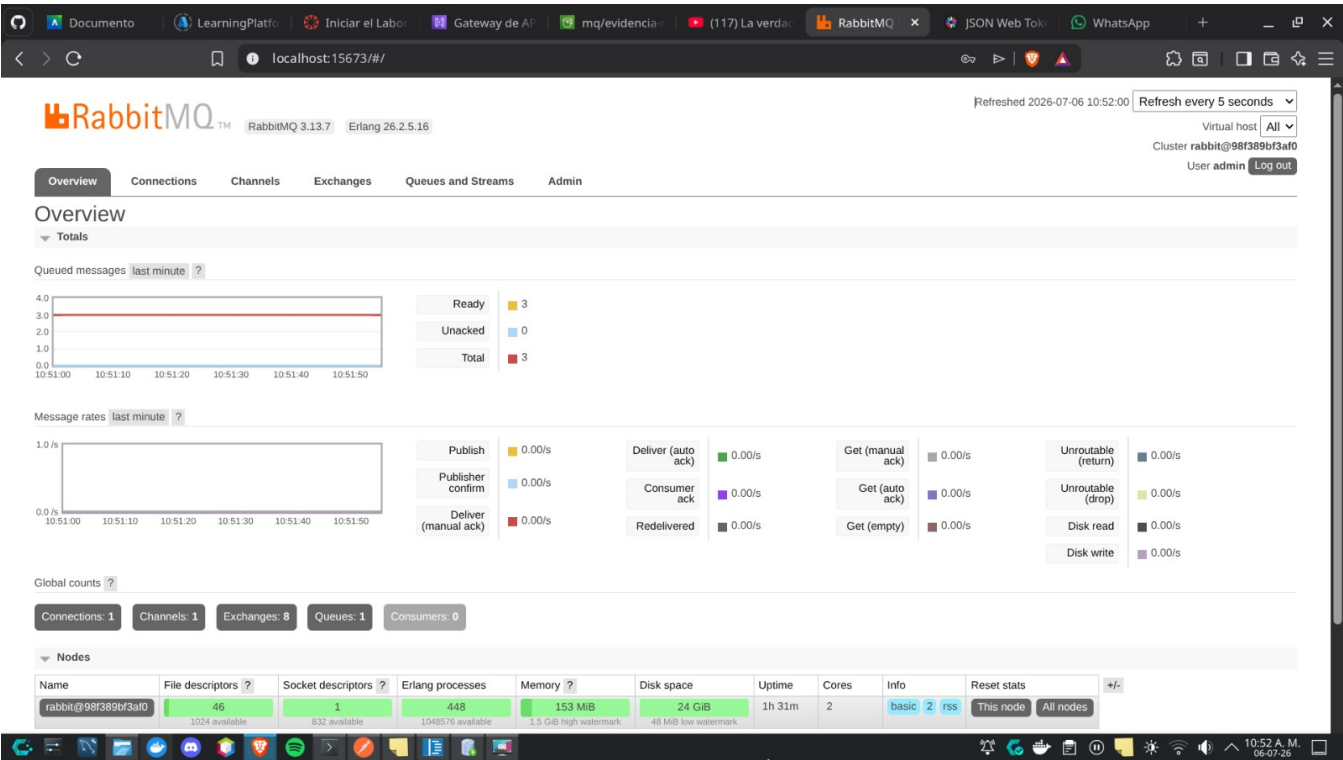


```
1 services:
2   rabbitmq:
3     image: rabbitmq:3-management
4     container_name: rabbitmq-learning
5     ports:
6       - "5672:5672"
7       - "15672:15672"
8     environment:
9       RABBITMQ_DEFAULT_USER: admin
10      RABBITMQ_DEFAULT_PASS: Admin12345
11     restart: "no"
12
13   learningplatformcloud:
14     image: ladyred/learningplatformcloud:latest
15     container_name: learningplatformcloud
16     depends_on:
17       - rabbitmq
18     ports:
19       - "8081:8080"
20     environment:
21       DB_URL: 'jdbc:oracle:thin:@semana1_high?TNS_ADMIN=/app/wallet'
22       DB_USERNAME: 'ADMIN'
23       DB_PASSWORD: 'OracleCloud123'
24
25     AWS_REGION: 'us-east-1'
26     AWS_S3_BUCKET_NAME: 'learningplatformcloud-grupo13'
27
28     AZURE_B2C_CLIENT_ID: ${AZURE_B2C_CLIENT_ID}
29     AZURE_B2C_ISSUER_URI: ${AZURE_B2C_ISSUER_URI}
30     AZURE_B2C_JWK_SET_URI: ${AZURE_B2C_JWK_SET_URI}
31
32     RABBITMQ_HOST: 'rabbitmq'
33     RABBITMQ_PORT: '5672'
34     RABBITMQ_USERNAME: 'admin'
```

5.2. Configuración y despliegue

RabbitMQ fue incorporado como servicio contenerizado dentro de la infraestructura desplegada en EC2. El broker utiliza el puerto 5672 para la comunicación AMQP con la aplicación y el puerto 15672 para su interfaz administrativa.

Para verificar el servicio desde el equipo local se utilizó un túnel SSH con redirección hacia localhost:15673. Esto permitió acceder al panel de administración y comprobar el estado del broker durante las pruebas.



5.3. Integración con Spring Boot

La integración se implementó mediante Spring AMQP y una topología compuesta por los siguientes recursos:

Recurso	Configuración
Exchange	learning.resumenes.exchange
Tipo	direct
Routing key	learning.resumenes.routing
Cola	learning.resumenes.queue

El backend publica cada resumen en el exchange utilizando la routing key configurada. El binding dirige el mensaje hacia learning.resumenes.queue, donde permanece disponible hasta su procesamiento.

Documento LearningPlatform Iniciar el Labor SecurityGroup mq/evidencia RabbitMQ x JSON Web Tok WhatsApp Reutilizar recu +

localhost:15673/#/exchanges/%2Flearning.resumenes.exchange

Refreshed 2026-07-06 10:59:40 Refresh every 5 seconds

Virtual host All Cluster rabbit@98f389bf3af0 User admin Log out

Overview Connections Channels Exchanges Queues and Streams Admin

Exchange: learning.resumenes.exchange

Overview

Message rates last minute ?

1.0 /s

0.0 /s

10:58:40 10:58:50 10:59:00 10:59:10 10:59:20 10:59:30

Publish (in) 0.00/s

Publish (Out) 0.00/s

Details

Type direct

Features durable: true

Policy

Bindings

This exchange

↓

To	Routing key	Arguments
learning.resumenes.queue	learning.resumenes.routing	Unbind

Add binding from this exchange

6. PRUEBAS FUNCIONALES EN POSTMAN

https://ht5blszp28.execute-api.us-east-1.amazonaws.com/DEV/api/inscripciones - Natalia Alvarado's Workspace

File Edit View Help

Search

Invite Upgrade

No environment

LearningPlatformCloud_Semana7 > https://ht5blszp28.execute-api.us-east-1.amazonaws.com/DEV/api/inscripciones

POST https://ht5blszp28.execute-api.us-east-1.amazonaws.com/DEV/api/inscripciones

Send

Docs Params Authorization Headers Body Scripts Settings

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

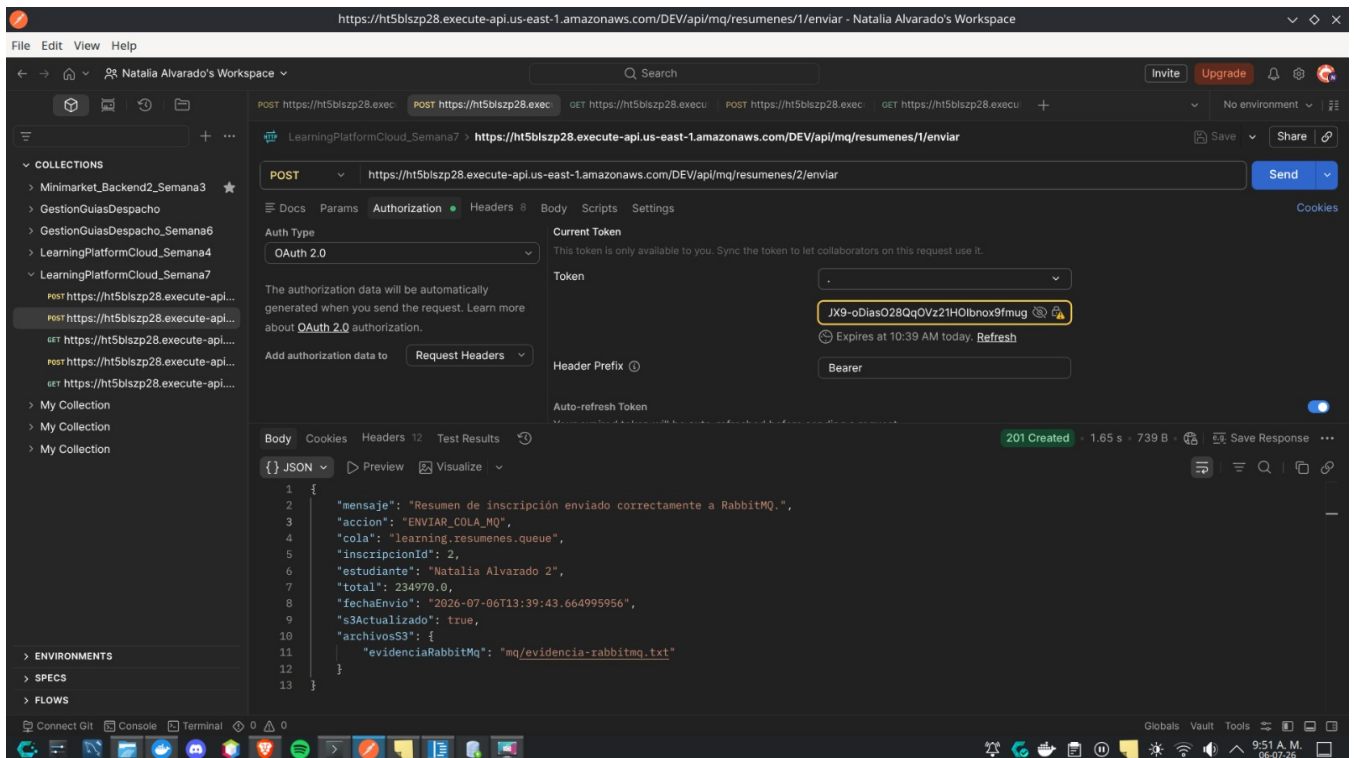
Body Cookies Headers 12 Test Results

201 Created - 2.65 s - 923 B Save Response

```
{
  "id": 2,
  "estudiante": "Natalia Alvarado 2",
  "cursos": [
    {
      "id": 3,
      "nombre": "Despliegue de Microservicios con Docker",
      "instructor": "Camila Torres",
      "duracionHoras": 28,
      "costo": 69990.0
    },
    {
      "id": 4,
      "nombre": "Persistencia Empresarial con Oracle Cloud",
      "instructor": "Javier Arancibia",
      "duracionHoras": 35,
      "costo": 79990.0
    },
    {
      "id": 5,
      "nombre": "Desarrollo Backend Profesional con Spring Data JPA",
      "instructor": "Valentina Espinoza",
      "duracionHoras": 30,
      "costo": 89990.0
    }
  ]
}
```

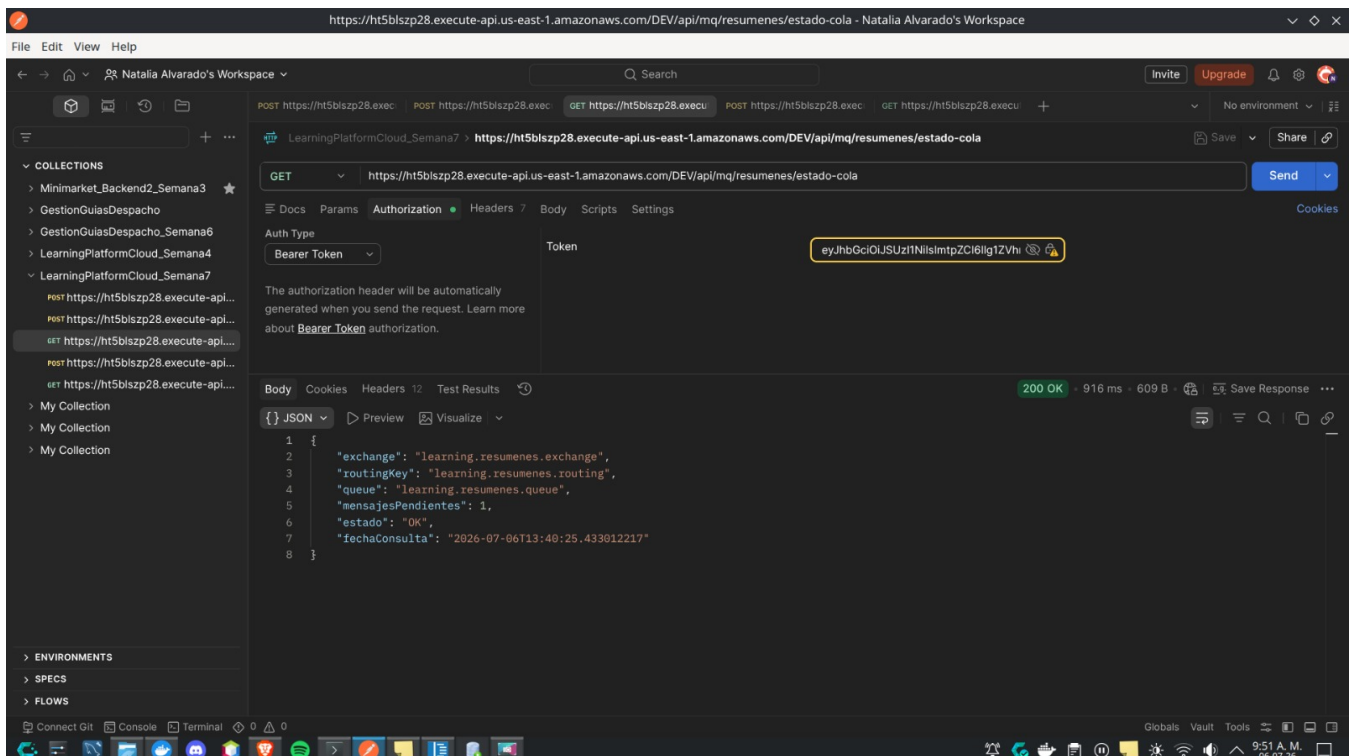
6.1. Envío de resúmenes

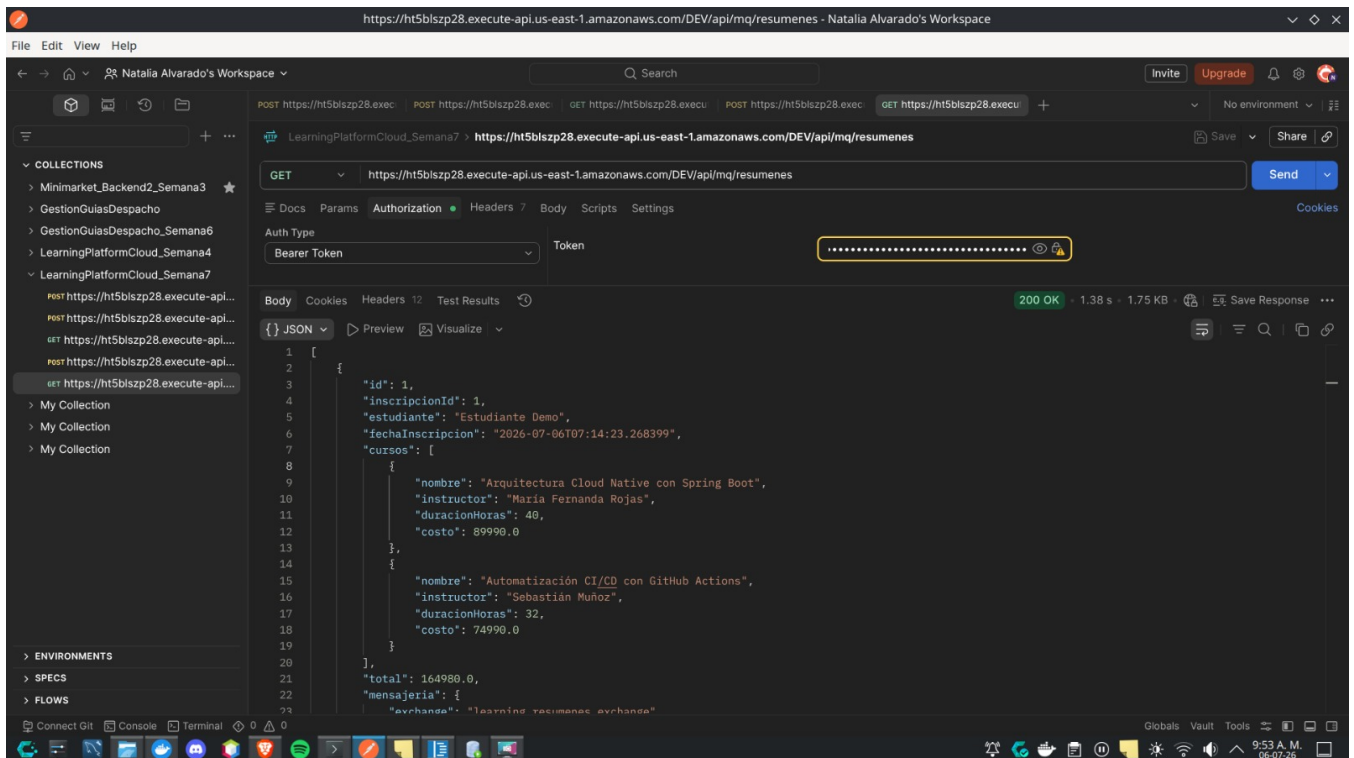
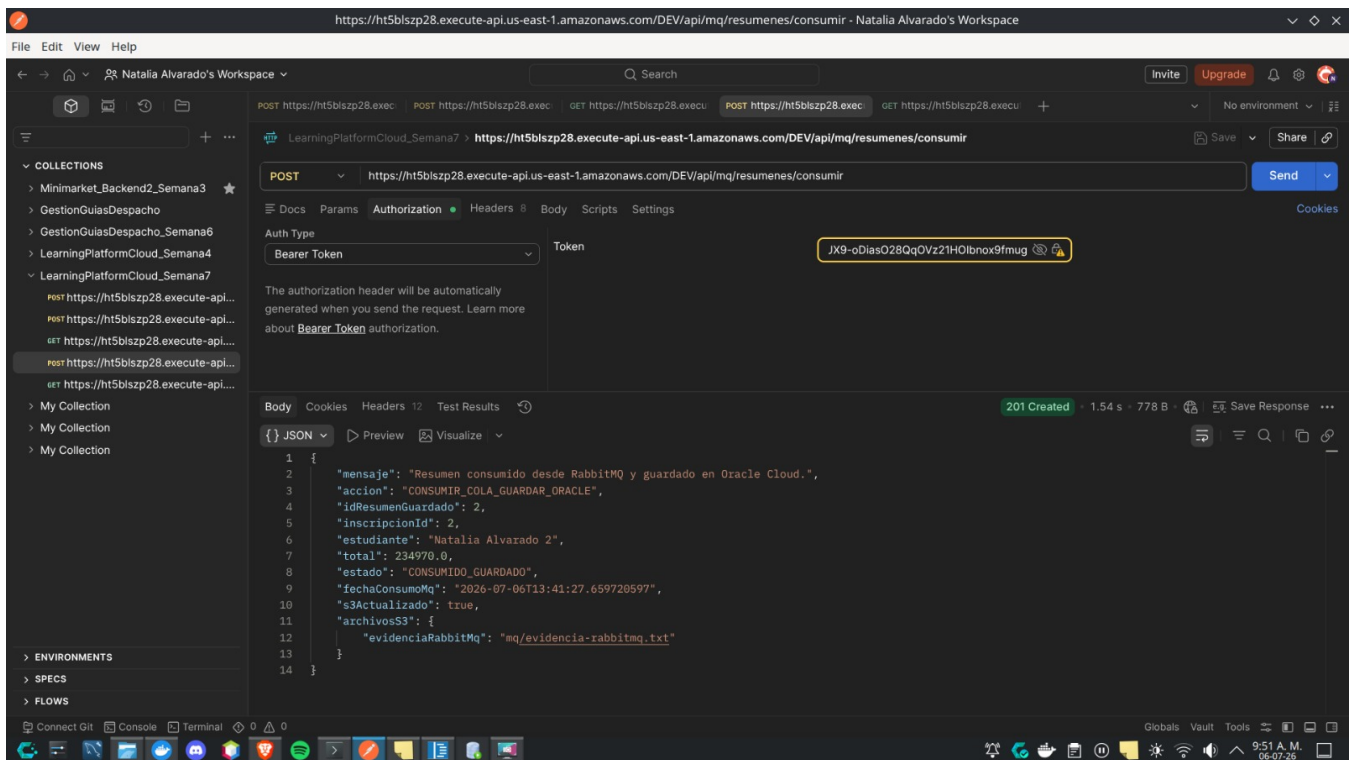
Las pruebas funcionales se ejecutaron sobre los endpoints publicados mediante API Gateway. El objetivo fue verificar el envío de resúmenes hacia RabbitMQ y comprobar posteriormente el estado de la cola.



La prueba fue realizada con distintas inscripciones para verificar que cada solicitud generara un mensaje independiente.

6.2. Consulta del estado de la cola





La operación se repitió con tres inscripciones diferentes para comprobar la acumulación independiente de mensajes en la cola.

7. VALIDACIÓN TÉCNICA EN RABBITMQ

Los resultados obtenidos en Postman fueron contrastados directamente con el broker para comprobar que la cantidad informada por la API correspondiera al estado real de la cola.

The screenshot shows the RabbitMQ Management UI in a web browser. The address bar indicates the URL is `localhost:15673/#/queues`. The page title is "RabbitMQ" and it shows the version "RabbitMQ 3.13.7" and "Erlang 26.2.5.16". The "Queues and Streams" tab is selected, showing a list of queues. The table below shows the details of the queue "learning.resumes.queue".

Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	learning.resumes.queue	classic	D	running	3	0	3	0.00/s	0.00/s	0.00/s

8. CONCLUSIONES

La implementación permitió incorporar RabbitMQ al proyecto LearningPlatformCloud y conectar el backend Spring Boot con una topología de mensajería compuesta por un exchange direct, una routing key y una cola durable.

Las pruebas realizadas mediante Postman confirmaron el envío independiente de tres resúmenes. Posteriormente, el endpoint de monitoreo informó tres mensajes pendientes, resultado que fue validado tanto en RabbitMQ Management como mediante una consulta directa al contenedor desplegado en EC2.

La coincidencia entre las cuatro evidencias obtenidas demuestra que las solicitudes fueron procesadas, enrutadas y almacenadas correctamente. Con ello, se verificó el funcionamiento de la integración entre API Gateway, Spring Boot y RabbitMQ dentro del entorno desplegado.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.